工业缺陷检测实训平台

开发指导书

产品版本 V4.0

文档版本 02

AWCloud

海云捷迅专有和保密信息
版权所有© 重庆海云捷迅科技有限公司

版权声明

版权所有©重庆海云捷迅科技有限公司 2024。保留一切权利。

未得到本公司的书面许可，任何人不得以任何方式或形式对本手册内的任何部分进行复制、摘录、备份、修改、传播、翻译成其他语言，将其全部或部分用于商业用途。

免责声明

本手册内容依据现有信息制作，由于产品版本升级或其他原因，其内容有可能变更。海云捷迅保留在没有任何通知或者提示的情况下对手册内容进行修改的权利。本手册仅作为使用指导，海云捷迅在编写本手册时已尽力保证其内容准确可靠，但并不确保手册内容完全没有错误或遗漏，本手册中的所有信息也不构成任何明示或暗示的担保。

重庆海云捷迅科技有限公司

地址：重庆市沙坪坝区学城大道 62-1 号研发楼一期 B3 栋 邮编：401332

网址：https://www.awcloud.com

邮箱：support@awcloud.com

电话：400-668-3172

传真：023-61736500

AWCloud

前言

读者对象

本手册适合下列人员阅读：

- 产品用户
- 技术支持工程师
- 售前工程师

图形界面元素引用约定

格式	意义
“”	带双引号“”的格式表示各类界面控件名称、数据输入或内容引用等，如单击“确定”。
>	多级菜单用“ > ”隔开。如选择“资源 > 物理机”，表示选择“资源”菜单下的“物理机”子菜单项。

标志约定

本手册还采用各种醒目标志来表示在操作过程中应该特别注意的地方，这些标志的意义如下：

标志	意义
粗体	命令行以粗体样式标识。

举例说明

- 本手册举例说明部分的端口类型同实际可能不符，实际操作中需要按照各产品所支持的端口类型进行操作。
- 本手册部分举例的显示信息中可能含有其它产品系列的内容（如产品型号、描述等），具体显示信息请以实际使用的设备信息为准。

修改记录

修订记录累积了每次文档更新的说明。最新版本的文档包含以前所有版本文档的更新内容。

- 文档版本 01 (2024-01-23)

第一次正式发布。

AwCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

目录

1 产品概述 1
2 功能架构 1
3 功能介绍 2
4 模型产出 2
4.1 环境准备 2
4.2 数据集准备 3
4.3 模型训练 8
4.4 模型导出 9
4.5 模型转换 9
5 FPGA 开发板推理 10
5.1 利用 PADDLE-LITE 推理 10
5.2 封装推理模型服务 15
5.3 FPGA 开发板推理 17
6 OPENCV 图像基础识别 18
6.1 基本操作 18
6.2 图像变换 19
6.3 图像平滑 20
6.3.1 噪声 20
6.3.2 卷积和滤波 22
6.3.3 均值滤波 26
6.3.4 方框滤波 26
6.3.5 中值滤波 27
6.3.6 高斯滤波 29
6.3.7 滤波器比较 31
7 机械臂控制 31
7.1 机械臂单个舵机控制 31
7.2 机械臂多个舵机同时控制 32
7.3 机械臂单个舵机位置读取 33
7.4 机械臂吸盘控制 34
8 SQLITE 使用 34
9 对外接口封装 35

AWCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

10 基于 VUE 的应用开发 36

10.1 VUE 基础实验 36

10.2 VUE 接口调用 38

AWCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

1 产品概述

工业缺陷检测实训平台是以工业互联网铝片表面缺陷检测应用为背景，面向高校人工智能、FPGA 等技术的实训产品。为贴合真实应用场景，通过加入专业的工业相机、光源、传输履带、机械臂形成一套完整的智能检测应用系统，实现基于 FPGA 的数据推理、机械臂自动分拣和检测数据可视化展示等功能。产品利用深度学习算法 SSD 和铝板表面缺陷数据集实现，并使用 FPGA 来运行模型应用可检测铝表面褶皱、脏污等缺陷类型，用户可二次开发增加缺陷种类。同时，结合教学内容与实际产业、岗位需求，该实训产品提供了丰富的实验课程及配套资源，让学生快速掌握相关技能知识，加强对行业需求的理解和认知。

2 功能架构

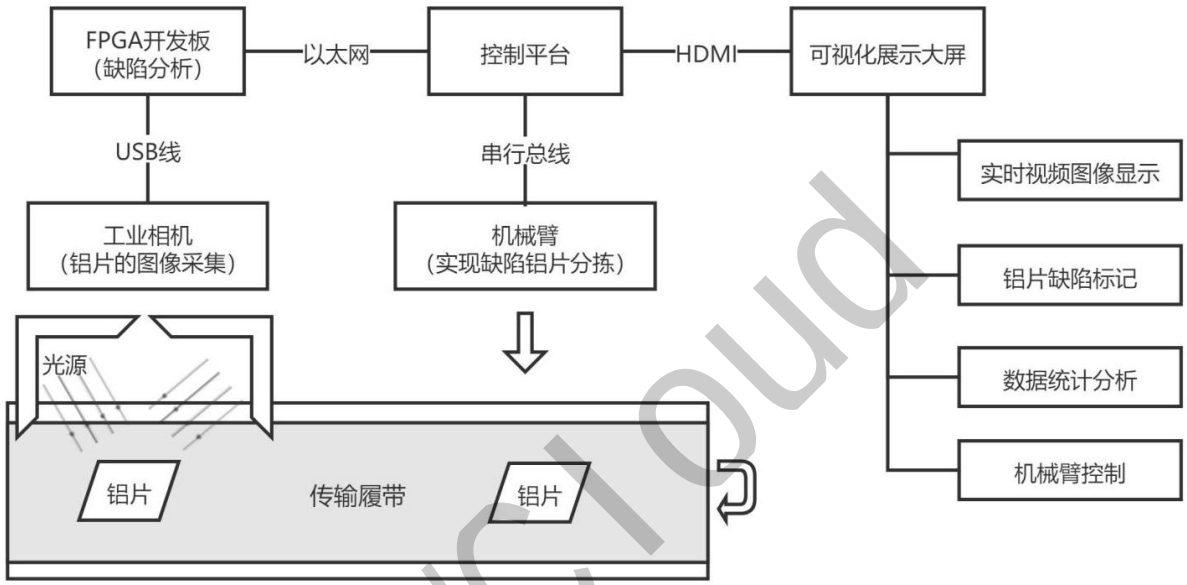


图 2-1 系统结构

【传输履带】：模拟工厂铝材产线履带传送过程，实现被检测铝片的传输。

【工业相机】：采用 600 万像素 1/1.8"CMOS USB3.0 工业面阵相机，通过 USB 传输方式与控制终端设备实现连接，实现被检测铝片的图像采集功能。

【光源】：采用亮度可调节条形光源，通过调节光源亮度，辅助实现铝片表现图像采集。

【机械臂】：采用 AI 视觉机械臂解决方案，抓手为吸盘结构。当平台检测到有缺陷的铝片时，机械臂自动从传送带上将有缺陷铝片抓取出来，实训缺陷铝片的自动分拣功能。

【控制终端】：用于控制缺陷检测的整体流程，包括基于 FPGA 的铝片缺陷检测服务、缺陷检测应用系统、机械臂控制服务。

【FPGA 开发板】：在 FPGA 开发板上运行铝片缺陷检测推理服务，实现对铝片缺陷种类及位置的判断，并将结果反馈给控制终端。

【显示器】：主要包括相机的视频流的实时显示，检测的铝片缺陷标记、数据的统计分析和机械臂的控制等几个部分。

【报警器】：当系统检测到缺陷铝片后通过报警器进行报警提示。

说明：开发板和机械臂 ip 配置为 172.16.68.0/23 网段。其中机械臂默认 ip 为：172.16.68.111/23，FPGA 开发板的 ip 默认为 172.16.68.111/23。

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

3 功能介绍

(1) 基于 FPGA 的缺陷检测

FPGA 设备利用 SSD 深度学习算法和铝片表面缺陷数据集训练的模型进行推理。当相机采集到视频流图片后，会发送给推理服务获取推理结果，并将缺陷情况记录到数据库，同时对外提供检测情况的统计分析数据接口。

(2) 机械臂控制

系统采用 5 轴吸盘结构的机械臂，可通过平台实现机械臂的左右转向、抓取/释放等功能控制，同时可以结合缺陷检测应用实现缺陷铝片的自动分拣功能。

(3) 铝片表面缺陷检测可视化展示

通过平台页面实时展示当前检测铝片的原始图片和视频流图像，并对有缺陷的铝片进行缺陷点位标记和种类标注。对检测的历史数据进行精准的质量统计，实现产品检测良品率、检测数量的统计，有效保证产品质量。

(4) 传输履带及光源

为了模拟工厂铝材缺陷检测真实场景，系统配套的传输履带的速度和环形光源亮度均可调节。

4 模型产出

4.1 环境准备

在进行模型训练前，需要先准备好模型训练环境，这里选用的 PaddleDetection 项目进行的模型训练。

环境要求：

ubuntu 18.04

python == 3.6 （如果条件不满足，可以考虑使用 conda 安装）

CUDA(must include the cudnn) Version: 11.0

依赖安装：

```
tar xf PaddelDetection.tar.gz
cd PaddleDetection
pip install -r requirements.txt
pip install paddlepaddle-gpu==2.1.3.post110 paddleslim==2.1.0 paddlelite==2.10 -f https://www.paddlepaddle.org.cn/whl/linux/mkl/avx/stable.html
pip install torch==1.7.1+cu110 -f https://download.pytorch.org/whl/cu110/torch_stable.html
# 验证 paddlepaddle 的安装
# python -c "import paddle;paddle.utils.run_check()"
# 如果出现错误，极有可能是由于 cuda 版本有问题
```

应用 paddlepaddle 补丁：

```
cd <paddlepaddle install dir>
# for example: cd /root/anaconda3/envs/paddle-2.1/lib/python3.6/site-packages/paddle
```

AWCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

```
3.
4.  patch -p0 < <install dir>/paddle.patch
5.  # for example, if the paddle.patch is in /root
6.  # patch -p0 < /root/paddle.patch
```

安装 PaddleDetection:

```
1.  python setup.py install
```

4.2 数据集准备

常用的制作数据集的软件有 labelme，labelimg 等，这里主要介绍 labelimg 的使用。这里需要用到 ubuntu 桌面

安装 labelimg：

```
1.  git clone https://github.com/tzutalin/labelImg.git
2.  cd labelImg
3.  apt-get install pyqt5-dev-tools
4.  pip3 install --upgrade pip
5.  pip3 install -r requirements/requirements-linux-python3.txt
6.  make qt5py3
```

labelimg 使用：

安装完 labelimg 后，下面介绍如何使用 labelimg

执行下面命令，打开 labelimg 软件

```
1.  python3 labelImg.py
```

执行上命令后，会自动弹出 labelimg 的界面，如下图所示



其中：

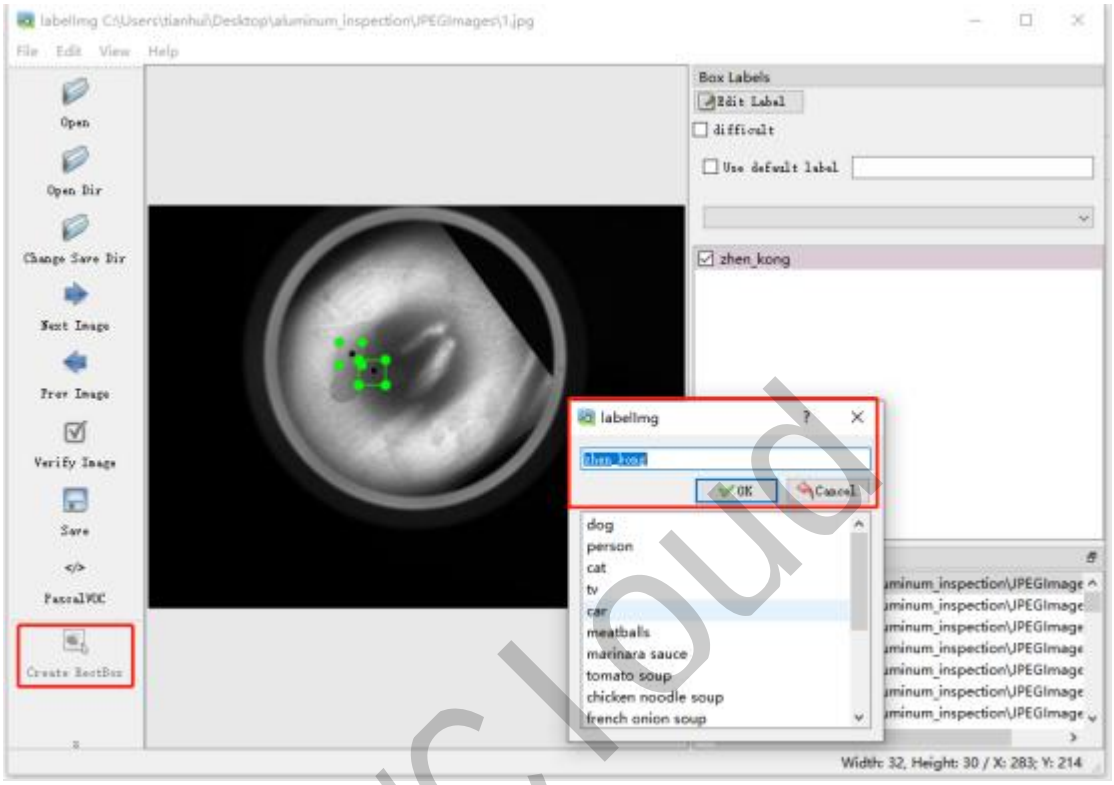
`Open Dir:` 打开存放原文件的文件夹，JPEGImages 文件夹。

AWCloud

`Change Save Dir：`用于选择标签文件存放的位置，选择 Annotations 文件夹。

在 view 菜单栏中打开 `auto save mode` 格式，这个可以帮助我们自动的保存标记好的图片。

点击 `creat RectBox` 开始标记，在 labelImg 中输入标签，如果感觉这个比较难检测，可以点击右上角的 `difficult`



快捷键：W（创建方框），A（上一张），D（下一张）。

标注完成后会得到一些标签：



xml 文件里面包含标注好图片的信息：

`folder：`图片所在目录

`filename：`图片名称

`path：`图片所在路径

`size：`图片大小

`object：`标注信息

`name：`标注类别

AWCloud

bndbox：标注框坐标

```
<annotation>
    <folder>Images</folder>
    <filename>1.jpg</filename>
    <path>C:\Users\tianhui\Desktop\aluminum_inspection\Images\1.jpg</path>
    <source>
        <database>Unknown</database>
    </source>
    <size>
        <width>640</width>
        <height>480</height>
        <depth>1</depth>
    </size>
    <segmented>0</segmented>
    <object>
        <name>zhen_kong</name>
        <pose>Unspecified</pose>
        <truncated>0</truncated>
        <difficult>0</difficult>
        <bndbox>
            <xmin>233</xmin>
            <ymin>157</ymin>
            <xmax>254</xmax>
            <ymax>193</ymax>
        </bndbox>
    </object>
    <object>
        <name>zhen_kong</name>
        <pose>Unspecified</pose>
        <truncated>0</truncated>
        <difficult>0</difficult>
        <bndbox>
            <xmin>256</xmin>
            <ymin>180</ymin>
            <xmax>282</xmax>
            <ymax>209</ymax>
        </bndbox>
    </object>
</annotation>
```

数据集制作：

利用 labelimg 标注好数据集图片后，需要对标注的图片进行进一步处理，才能作为训练需要的数据集使用

将图片与生成的 xml 文件分别存放在 images 和 annotations 目录，并创建一个 ImageSet 目录用来存放制作数据集时生成的临时文件，如下所示

```
my_dataset                      # 根目录
|-- annotations                  # xml 文件目录
|    |-- xxx.xml                 # 生成的 xml 文件
|    |...
|-- images                       # 图像目录
|    |-- xxx.jpg(png or other)   # 图片
```

AWCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

```
|    |...
|-- label_list.txt              # 数据集的类别名称
|-- ImageSet                  # 用于生成对应 txt 文件临时目录
```

使用 dataset_processing_one.py 对数据集进行第一步处理

```python
import os
import random

trainval_percent = 0.95  #训练集验证集总占比
train_percent = 0.9  #训练集在 trainval_percent 里的 train 占比
xmlfilepath = './annotations'
txtsavepath = './images'
total_xml = os.listdir(xmlfilepath)

num=len(total_xml)
list=range(num)
tv=int(num*trainval_percent)
tr=int(tv*train_percent)
trainval= random.sample(list,tv)
train=random.sample(trainval,tr)

ftrainval = open('./ImageSet/trainval.txt', 'w')
ftest = open('./ImageSet/test.txt', 'w')
ftrain = open('./ImageSet/train.txt', 'w')
fval = open('./ImageSet/val.txt', 'w')

for i  in list:
    name=total_xml[i][:-4]+'\n'
    if i in trainval:
        ftrainval.write(name)
        if i in train:
            ftrain.write(name)
        else:
            fval.write(name)
    else:
        ftest.write(name)

ftrainval.close()
ftrain.close()
fval.close()
ftest .close()
```

再使用 dataset_processing_two.py 对数据集进行进一步处理

```python
import os
import re

devkit_dir = './'
output_dir = './'

```

AWCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

```
def get_dir(devkit_dir,  type):
    return os.path.join(devkit_dir, type)

def walk_dir(devkit_dir):
    filelist_dir = get_dir(devkit_dir, 'ImageSet')
    annotation_dir = get_dir(devkit_dir, 'annotations')
    img_dir = get_dir(devkit_dir, 'images')
    trainval_list = []
    train_list = []
    val_list = []
    test_list = []

    added = set()

    for _, _, files in os.walk(filelist_dir):
        for fname in files:
            print(fname)
            img_ann_list = []
            if re.match('trainval.txt', fname):
                img_ann_list = trainval_list
            elif re.match('train.txt', fname):
                img_ann_list = train_list
            elif re.match('val.txt', fname):
                img_ann_list = val_list
            elif re.match('test.txt', fname):
                img_ann_list = test_list
            else:
                continue
            fpath = os.path.join(filelist_dir, fname)
            for line in open(fpath):
                name_prefix = line.strip().split()[0]
                print(name_prefix)

                added.add(name_prefix)
                #ann_path = os.path.join(annotation_dir, name_prefix + '.xml')
                ann_path = annotation_dir + '/' + name_prefix + '.xml'
                print(ann_path)
                #img_path = os.path.join(img_dir, name_prefix + '.jpg')
                img_path = img_dir + '/' + name_prefix + '.jpg'
                assert os.path.isfile(ann_path), 'file %s not found.' % ann_pa
th
                assert os.path.isfile(img_path), 'file %s not found.' % img_pa
th
                img_ann_list.append((img_path, ann_path))
            print(img_ann_list)

    return trainval_list, train_list, val_list, test_list


```

AWCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

```python
def prepare_filelist(devkit_dir, output_dir):
    trainval_list = []
    train_list = []
    val_list = []
    test_list = []

    trainval, train, val, test = walk_dir(devkit_dir)

    trainval_list.extend(trainval)
    train_list.extend(train)
    val_list.extend(val)
    test_list.extend(test)
    #print(trainval)
    with open(os.path.join(output_dir, 'trainval.txt'), 'w') as ftrainval:
        for item in trainval_list:
            ftrainval.write(item[0] + ' ' + item[1] + '\n')

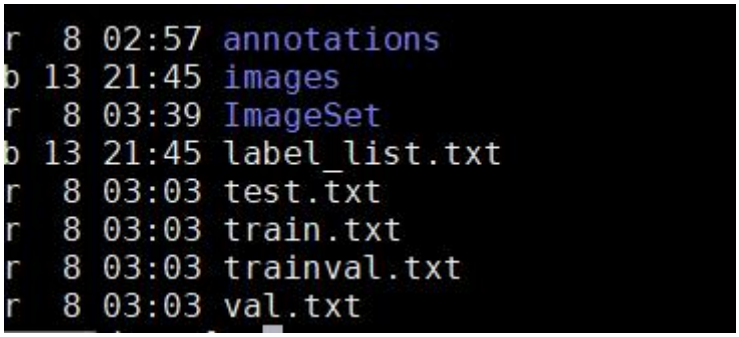
    with open(os.path.join(output_dir, 'train.txt'), 'w') as ftrain:
        for item in train_list:
            ftrain.write(item[0] + ' ' + item[1] + '\n')

    with open(os.path.join(output_dir, 'val.txt'), 'w') as fval:
        for item in val_list:
            fval.write(item[0] + ' ' + item[1] + '\n')

    with open(os.path.join(output_dir, 'test.txt'), 'w') as ftest:
        for item in test_list:
            ftest.write(item[0] + ' ' + item[1] + '\n')


if __name__ == '__main__':
    prepare_filelist(devkit_dir, output_dir)
```

通过处理后，可以得到能够进行训练使用的数据集。



4.3 模型训练

配置文件修改：

```
vim configs/datasets/voc.yml
```

AWCloud

```
metric: VOC
map_type: 11point
num_classes: 4        缺陷类型总数

TrainDataset:
  !VOCDataSet
    dataset_dir: dataset/aluminum_ssd_voc        数据集路径
    anno_path: trainval.txt
    label_list: label_list.txt
    data_fields: ['image', 'gt_bbox', 'gt_class', 'difficult']

EvalDataset:
  !VOCDataSet
    dataset_dir: dataset/aluminum_ssd_voc
    anno_path: test.txt
    label_list: label_list.txt
    data_fields: ['image', 'gt_bbox', 'gt_class', 'difficult']

TestDataset:
  !ImageFolder
    anno_path: dataset/aluminum_ssd_voc/label_list.txt
~
~
```

全精度训练：

```
python tools/train.py -c configs/ssd/ssd_mobilenet_v1_300_120e_voc.yml --eval
```

量化训练：

修改 configs/slim/quant/ssd_mobilenet_v1_qat.yml，将 pretrain_weights 参数指向

ssd_mobilenet_v1_300_120e_voc 文件所在路径

例如

pretrain_weights: output/ssd_mobilenet_v1_300_120e_voc/best_model

```
python tools/train.py -c configs/ssd/ssd_mobilenet_v1_300_120e_voc.yml --
slim_config configs/slim/quant/ssd_mobilenet_v1_qat.yml --eval
```

4.4 模型导出

最终导出的静态模型位于 output_inference/ssd_mobilenet_v1_qat/

```
python tools/export_model.py \
    -c configs/ssd/ssd_mobilenet_v1_300_120e_voc.yml \
    --slim_config configs/slim/quant/ssd_mobilenet_v1_qat.yml \
    -o weights=output/ssd_mobilenet_v1_qat/best_model
```

4.5 模型转换

当前目录则会生成 ssd_mobilenet_v1_opt.nb，该文件即是量化之后的模型

```
opt \
```

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

```
    --model_dir=./PaddleDetection/output_inference/ssd_mobilenet_v1_qat/ \
    --valid_targets=intel_fpga,arm \
    --optimize_out_type=naive_buffer \
    --optimize_out=ssd_mobilenet_v1_opt
```

5 FPGA 开发板推理

5.1 利用 Paddle-Lite 推理

在 FPGA 上推理，这里使用 Paddle-Lite，下面针对 Paddle-Lite 推理流程进行简要说明

C++代码调用 Paddle Lite 执行预测库仅需以下五步：

(1) 引用头文件和命名空间

```
#include "paddle_api.h"using namespace paddle::lite_api;
```

(2) 指定模型文件，创建 Predictor

```
// 1. Set MobileConfig

MobileConfig config;

// 2. Set the path to the model generated by opt tools

config.set_model_from_file(model_file_path);

// 3. Create PaddlePredictor by MobileConfig

std::shared_ptr<PaddlePredictor> predictor= CreatePaddlePredictor<MobileConfig>(config);
```

(3) 设置模型输入 (下面以全一输入为例)

```
std::unique_ptr<Tensor> input_tensor(std::move(predictor->GetInput(0)));

input_tensor->Resize({1, 3, 224, 224});

auto* data = input_tensor->mutable_data<float>();

for (int i = 0; i < ShapeProduction(input_tensor->shape()); ++i) {

  data[i] = 1;

}
```

如果模型有多个输入，每一个模型输入都需要准确设置 shape 和 data。

(4) 执行预测

```
predictor->Run();
```

(5) 获得预测结果

```
std::unique_ptr<const Tensor> output_tensor(
```

AWCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

```
    std::move(predictor->GetOutput(0)));

// 转化为数据

auto output_data=output_tensor->data<float>();
```

下面给个具体示例，使用 SSD 模型进行推理

```
#include <iostream>
#include <vector>
#include "opencv2/core.hpp"
#include "opencv2/imgcodecs.hpp"
#include "opencv2/imgproc.hpp"
#include "paddle_api.h"  // NOLINT

using namespace paddle::lite_api;  // NOLINT

struct Object {
  int batch_id;
  cv::Rect rec;
  int class_id;
  float prob;
};

int64_t ShapeProduction(const shape_t& shape) {
  int64_t res = 1;
  for (auto i : shape) res *= i;
  return res;
}

const char* class_names[] = {
    "background", "aeroplane",   "bicycle", "bird",  "boat",
    "bottle",     "bus",         "car",     "cat",   "chair",
    "cow",        "diningtable", "dog",     "horse", "motorbike",
    "person",     "pottedplant", "sheep",   "sofa",  "train",
    "tvmonitor"};

// fill tensor with mean and scale and trans layout: nhwc -
   > nchw, neon speed up
void neon_mean_scale(const float* din,
                     float* dout,
                     int size,
                     const std::vector<float> mean,
                     const std::vector<float> scale) {
  if (mean.size() != 3 || scale.size() != 3) {
    std::cerr << "[ERROR] mean or scale size must equal to 3\n";
    exit(1);
  }
  float32x4_t vmean0 = vdupq_n_f32(mean[0]);
  float32x4_t vmean1 = vdupq_n_f32(mean[1]);
  float32x4_t vmean2 = vdupq_n_f32(mean[2]);
```

AWCloud

```
  float32x4_t vscale0 = vdupq_n_f32(1.f / scale[0]);
  float32x4_t vscale1 = vdupq_n_f32(1.f / scale[1]);
  float32x4_t vscale2 = vdupq_n_f32(1.f / scale[2]);

  float* dout_c0 = dout;
  float* dout_c1 = dout + size;
  float* dout_c2 = dout + size * 2;

  int i = 0;
  for (; i < size - 3; i += 4) {
    float32x4x3_t vin3 = vld3q_f32(din);
    float32x4_t vsub0 = vsubq_f32(vin3.val[0], vmean0);
    float32x4_t vsub1 = vsubq_f32(vin3.val[1], vmean1);
    float32x4_t vsub2 = vsubq_f32(vin3.val[2], vmean2);
    float32x4_t vs0 = vmulq_f32(vsub0, vscale0);
    float32x4_t vs1 = vmulq_f32(vsub1, vscale1);
    float32x4_t vs2 = vmulq_f32(vsub2, vscale2);
    vst1q_f32(dout_c0, vs0);
    vst1q_f32(dout_c1, vs1);
    vst1q_f32(dout_c2, vs2);

    din += 12;
    dout_c0 += 4;
    dout_c1 += 4;
    dout_c2 += 4;
  }
  for (; i < size; i++) {
    *(dout_c0++) = (*(din++) - mean[0]) * scale[0];
    *(dout_c1++) = (*(din++) - mean[1]) * scale[1];
    *(dout_c2++) = (*(din++) - mean[2]) * scale[2];
  }
}

void pre_process(const cv::Mat& img, int width, int height, float* data) {
  cv::Mat rgb_img;
  cv::cvtColor(img, rgb_img, cv::COLOR_BGR2RGB);
  cv::resize(rgb_img, rgb_img, cv::Size(width, height), 0.f, 0.f);
  cv::Mat imgf;
  rgb_img.convertTo(imgf, CV_32FC3, 1 / 255.f);
  std::vector<float> mean = {0.5f, 0.5f, 0.5f};
  std::vector<float> scale = {0.5f, 0.5f, 0.5f};
  const float* dimg = reinterpret_cast<const float*>(imgf.data);
  neon_mean_scale(dimg, data, width * height, mean, scale);
}

std::vector<Object> detect_object(const float* data,
                                  int count,
                                  float thresh,
                                  cv::Mat& image) {
```

AWCloud

```
92.   if (data == nullptr) {
93.     std::cerr << "[ERROR] data can not be nullptr\n";
94.     exit(1);
95.   }
96.   std::vector<Object> rect_out;
97.   for (int iw = 0; iw < count; iw++) {
98.     int oriw = image.cols;
99.     int orih = image.rows;
100.      if (data[1] > thresh && static_cast<int>(data[0]) > 0) {
101.        Object obj;
102.        int x = static_cast<int>(data[2] * oriw);
103.        int y = static_cast<int>(data[3] * orih);
104.        int w = static_cast<int>(data[4] * oriw) - x;
105.        int h = static_cast<int>(data[5] * orih) - y;
106.        cv::Rect rec_clip =
107.            cv::Rect(x, y, w, h) & cv::Rect(0, 0, image.cols, image.rows);
108.        obj.batch_id = 0;
109.        obj.class_id = static_cast<int>(data[0]);
110.        obj.prob = data[1];
111.        obj.rec = rec_clip;
112.        if (w > 0 && h > 0 && obj.prob <= 1) {
113.          rect_out.push_back(obj);
114.          cv::rectangle(image, rec_clip, cv::Scalar(0, 0, 255), 2, cv::LINE_AA)
    ;
115.          std::string str_prob = std::to_string(obj.prob);
116.          std::string text = std::string(class_names[obj.class_id]) + ": " +
117.                             str_prob.substr(0, str_prob.find(".") + 4);
118.          int font_face = cv::FONT_HERSHEY_COMPLEX_SMALL;
119.          double font_scale = 1.f;
120.          int thickness = 2;
121.          cv::Size text_size =
122.              cv::getTextSize(text, font_face, font_scale, thickness, nullptr);
123.          float new_font_scale = w * 0.35 * font_scale / text_size.width;
124.          text_size = cv::getTextSize(
125.              text, font_face, new_font_scale, thickness, nullptr);
126.          cv::Point origin;
127.          origin.x = x + 10;
128.          origin.y = y + text_size.height + 10;
129.          cv::putText(image,
130.                      text,
131.                      origin,
132.                      font_face,
133.                      new_font_scale,
134.                      cv::Scalar(0, 255, 255),
135.                      thickness,
136.                      cv::LINE_AA);
137.
138.          std::cout << "detection, image size: " << image.cols << ", "
139.                    << image.rows
```

AWCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

```
                    << ", detect object: " << class_names[obj.class_id]
                    << ", score: " << obj.prob << ", location: x=" << x
                    << ", y=" << y << ", width=" << w << ", height=" << h
                    << std::endl;
        }
      }
      data += 6;
    }
    return rect_out;
}

void RunModel(std::string model_file, std::string img_path) {
    // 1. Set MobileConfig
    MobileConfig config;
    config.set_model_from_file(model_file);

    // 2. Create PaddlePredictor by MobileConfig
    std::shared_ptr<PaddlePredictor> predictor =
        CreatePaddlePredictor<MobileConfig>(config);

    // 3. Prepare input data from image
    std::unique_ptr<Tensor> input_tensor(std::move(predictor->GetInput(0)));
    const int in_width = 300;
    const int in_height = 300;
    input_tensor->Resize({1, 3, in_height, in_width});
    auto* data = input_tensor->mutable_data<float>();
    cv::Mat img = imread(img_path, cv::IMREAD_COLOR);
    pre_process(img, in_width, in_height, data);

    // 4. Run predictor
    predictor->Run();

    // 5. Get output and post process
    std::unique_ptr<const Tensor> output_tensor(
        std::move(predictor->GetOutput(0)));
    auto* outptr = output_tensor->data<float>();
    auto shape_out = output_tensor->shape();
    int64_t cnt = ShapeProduction(shape_out);
    auto rec_out = detect_object(outptr, static_cast<int>(cnt / 6), 0.6f, img);
    int start = img_path.find_last_of("/");
    int end = img_path.find_last_of(".");
    std::string img_name = img_path.substr(start + 1, end - start - 1);
    std::string result_name = img_name + "_ssd_detection_result.jpg";
    cv::imwrite(result_name, img);
}

int main(int argc, char** argv) {
    if (argc < 3) {
      std::cerr << "[ERROR] usage: " << argv[0] << " model_file image_path\n";
```

AWCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

```
    exit(1);
  }
  std::string model_file = argv[1];
  std::string img_path = argv[2];
  RunModel(model_file, img_path);
  return 0;
}
```

5.2 封装推理模型服务

Paddle-Lite 推理程序使用的 C++语言进行编写，需要在原有的程序上进行扩展，使得外部能够使用该程序进行模型推理，因此需要了解 C++如何提供 http 服务。

这里使用了 github 项目上提供的 httplib 库，具体 httplib 相关用法可以查看 https://github.com/yhirose/cpp-httplib， 下面简单介绍下 httplib 的使用

```
#include "httplib.h"

using namespace httplib;


// 请求日志打印
std::string log(const Request &req, const Response &res) {
  std::string s;
  char buf[BUFSIZ];

  s += "================================\n";

  snprintf(buf, sizeof(buf), "%s %s %s", req.method.c_str(),
           req.version.c_str(), req.path.c_str());
  s += buf;

  std::string query;
  for (auto it = req.params.begin(); it != req.params.end(); ++it) {
    const auto &x = *it;
    snprintf(buf, sizeof(buf), "%c%s=%s",
             (it == req.params.begin()) ? '?' : '&', x.first.c_str(),
             x.second.c_str());
    query += buf;
  }
  snprintf(buf, sizeof(buf), "%s\n", query.c_str());
  s += buf;

  s += dump_headers(req.headers);

  s += "--------------------------------\n";

  snprintf(buf, sizeof(buf), "%d %s\n", res.status, res.version.c_str());
  s += buf;
  s += dump_headers(res.headers);
```

AWCloud

```
    s += "\n";

    if (!res.body.empty()) { s += res.body; }

    s += "\n";

    return s;
}

// html 页面
const char *html = R"(

<form id="formElem">
  <input type="file" name="image_file" accept="image/*">
  <input type="submit">
</form>
<script>
  formElem.onsubmit = async (e) => {
    e.preventDefault();
    let res = await fetch('/post', {
      method: 'POST',
      body: new FormData(formElem)
    });
    console.log(await res.text());
  };
</script>
)";

int main(int argc, char* argv[]){
    Server svr;
    // 一次只允许接收处理一个请求
    svr.new_task_queue = [] { return new ThreadPool(1); };
    // 定义GET 请求接口，获取 html 页面显示
    svr.Get("/", [](const Request & /*req*/, Response &res) {
      res.set_content(html, "text/html");
    });
    // 定义 POST 请求接口
    svr.Post("/predict", [](const Request &req, Response &res) {
   // 获取上传的图片文件
      auto image_file = req.get_file_value("image_file");
   // 定义Mat 格式数据，将图片信息写入其中，方便推理使用
      cv::Mat img_decode;
      std::vector<uchar> data(image_file.content.begin(), image_file.content.end());
      img_decode = cv::imdecode(data, CV_LOAD_IMAGE_COLOR);
      json k;
      std::ifstream is("config.json");
      is >> k;
   // 根据上传的图片获取推理结果
```

AWCloud

```
        std::string out_put = predict(k, img_decode);

        // POST 请求返回信息，这里返回推理结果
        res.set_content(out_put, "text/plain");

    });
// 请求日志显示
    svr.set_logger([](const Request &req, const Response &res) {
      printf("%s", log(req, res).c_str());
    });
// http 请求端口号定义
    svr.listen("0.0.0.0", 8080);
    return 0;
}
```

5.3 FPGA 开发板推理

下面是完整的使用 FPGA 开发板进行推理的流程

a.将 ssd_detection_demo.tar.gz 拷贝到 fpga 开发板/opt 目录，tar -xvf ssd_detection_demo.tar.gz

c.将 ssd_detection_demo/ssd_mobilenet_v1/ssd_mobilenet_v1_opt.nb 更换为 4.5 生成的 ssd_mobilenet_v1_opt.nb 模型文件

d.将 ssd_detection_demo/labels/label_list 更换为 4.2 中的 label_list 文件

e.编译

cd ssd_detection_demo/ssd_detection_src

./build.sh

f.将生成的 ssd_detection 文件拷贝到 ssd_detection_demo 目录

cp ssd_detection_src/build/ssd_detection ssd_detection_demo/

g.将推理程序做成服务

vim /etc/systemd/system/detect.service

```
[Unit]
Description = Api Service
After = sshd.service

[Service]
Type = simple
User = root
Group = root
ExecStart = /bin/sh -c "bash /opt/ssd_detection_demo/run.sh"

[Install]
WantedBy = multi-user.target
```

h.启动服务

AWC Ioud

systemctl enable detect.service

systemctl start detect.service

i.利用浏览器打开开发板 ip:8080，能够成功显示推理界面，可以通过浏览器查看推理效果



6 OpenCV 图像基础识别

作为计算机视觉应用最广泛的库，opencv 在图像处理中是绕不开的话题，本章节将做一部分 opencv 的基础功能介绍，以便在后续章节的图像处理中用 户对用到的 opencv 函数有基本的认识。opencv 有 C++和 Python 两个版本，为了更好的入门，本章节的教程使用 Python 版本。

6.1 基本操作

1. 读取图片

opencv 使用 cv2.imread(path, flags)来读入一张图片：

```
1. import cv2
2. # 加载灰度图
3. img = cv2.imread('lena.jpg', 0)
```

- 参数 path：图片的存放路径， 注意：图片路径中不能有中文字符。
- 参数 flags：根据图片的格式有三个读取方式，默认是彩色图，用法如下表所示。

flags	图片格式
cv2.IMREAD_COLOR	彩色图，默认值(1)
cv2.IMREAD_GRAYSCALE	灰度图(0)
cv2.IMREAD_UNCHANGED	包含透明通道的彩色图(-1)

2. 读取摄像头并显示

3. 要使用摄像头，需要使用 cv2.VideoCapture(path) 创建 VideoCapture 对象，传入的参数可以是摄像头的编号，也可以是视频文件，cv2.imshow()用来显示图像。

```
1. # 打开摄像头并显示
2. import cv2
3. capture = cv2.VideoCapture(0)
4. while(True):
5. # 获取一帧
6. ret, frame = capture.read()
7. cv2.imshow('frame', frame)
8. if cv2.waitKey(1) == ord('q'):
9. break
```

AWC loud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

- cv2.VideoCapture 参数 path ：摄像头的编号，0 代表第一个摄像头，也可以是视频文件，如 'test.mp4'
- cv2.imshow 参数：根据图片的格式有三个读取方式，默认是彩色图

4. 获取图片属性

img.shape 函数可以获取图像的属性，彩色图返回高度、宽度和通道数，灰度图只返回高度和宽度。

```
1. import cv2
2. # 加载灰度图
3. img = cv2.imread('lena.jpg', 0)
4. print(img.shape)
5. #打印 (480, 640, 3)，说明这是高 480 宽 640 的三通道图片
```

6.2 图像变换

1. 缩放

cv2.resize(src, dsize[, dst[, fx[, fy[, interpolation]]]]) 函数可以按照比例缩放图像，也可以按照指定的大小缩放图片，在图像的预处理 中经常要使用此函数。

```
1. import cv2
2. img = cv2.imread('drawing.jpg')
3. # 按照指定的宽度、高度缩放图片
4. res = cv2.resize(img, (132, 150))
5. # 按照比例缩放，如 x,y 轴均放大一倍
6. res2 = cv2.resize(img, None, fx=2, fy=2, interpolation=cv2.INTER_LINEAR)
7. cv2.imshow('shrink', res), cv2.imshow('zoom', res2)
8. cv2.waitKey(0)
```

- 参数 src ： 图片
- 参数 dsize ：缩放后的图像大小，用元组（宽度，高度）表示，在使用等比例缩放时这个参数为 None ， 如果指定参数 dsize 的值,则无论是否 指定了参数 fx 和 fy 的值,都由参数 dsize 来决定目标图像的大小
- 参数 fx ： 水平方向的缩放比例
- 参数 fy ： 竖直方向的缩放比例
- 参数 interpolation ： interpolation 代表插值方式，有三种值，默认是 INTER_LINEAR ，用法如下表所示。

interpolation 值	所用的插值方法
INTER_NEAREST	最近邻插值
INTER_LINEAR	双线性插值（默认设置）
INTER_AREA	使用像素区域关系进行重采样。 它可能是图像抽取的首选方法，因为它会产生无云纹理的结果。 但是当图像缩放 时，它类似于 INTER_NEAREST 方法。
INTER_CUBIC	4x4 像素邻域的双三次插值

AWCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

INTER_LANCZOS4	8x8 像素邻域的 Lanczos 插值

2. 镜像

dst = cv2.flip(img, flipcode)可以水平或竖直镜像图像

```
import cv2
img = cv2.imread('drawing.jpg')
# 水平翻转图片
dst = cv2.flip(img, 1)
```

- 参数 src：图片
- 参数 flipcode：代表翻转方式，有三个模式，用法如下表所示。

flipCode	dst
>0	水平翻转
=0	垂直翻转
<0	水平和垂直翻转

6.3 图像平滑

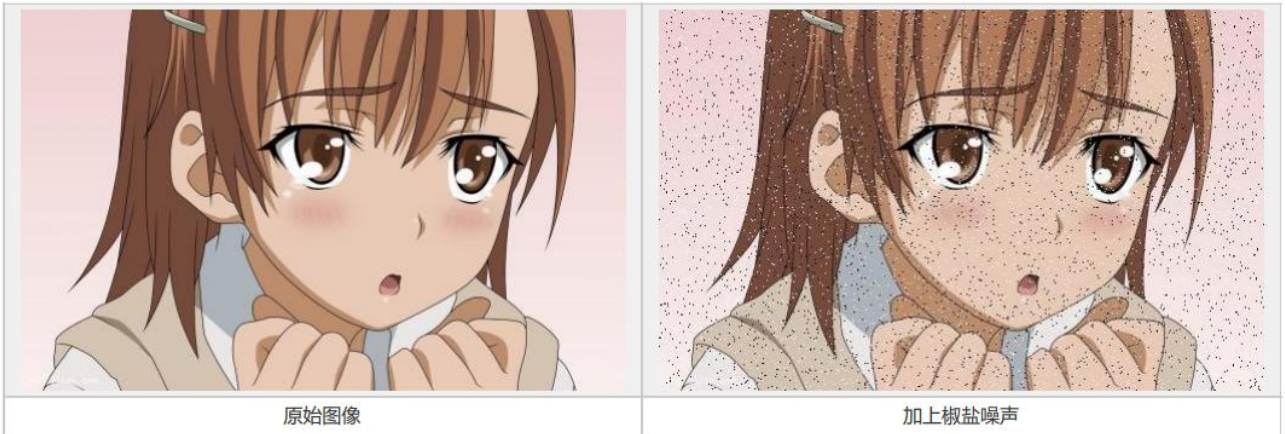
图像平滑也称为图像模糊(Blurring)，对图像使用低通滤波器可以去除图像中的高频成分（比如：噪音，边界)，用来降低噪声和伪影的影响。在讲解之前，我们需要介绍一些概念：噪声、卷积和卷积核。

6.3.1 噪声

6.3.1.1 椒盐噪声

椒盐噪声（salt-and-pepper noise）也称为脉冲噪声，它是一种随机出现的白点或者黑点，其中白点称为盐噪声，黑点称为椒噪声。前者是高灰度噪声，后者属于低灰度噪声。一般两种噪声同时出现，呈现在图像上就是黑白杂点。

下面是图像加上椒盐噪声的效果



原始图像　　加上椒盐噪声

AWCloud

6.3.1.2 高斯噪声

椒盐噪声是出现在随机位置、噪点灰度值基本固定的噪声，高斯噪声与其相反，是几乎每个点上都出现噪声、噪点灰度值的概率密度函数服从高斯分 布。高斯概率密度函数如下所示。

$$f(x)=\frac{1}{\sqrt{2\pi}\sigma}\exp(-\frac{(x-\mu)^2}{2\sigma^2})$$

几何上的高斯函数如下图所示，其中噪点的灰度值用 z 表示，灰度值的期望值用 μ 表示，灰度值的标准差用 σ 表示。

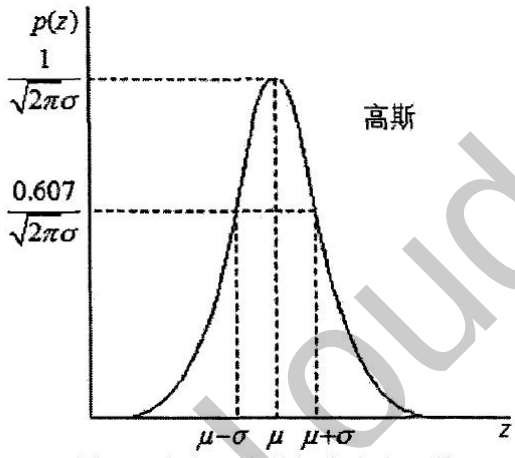
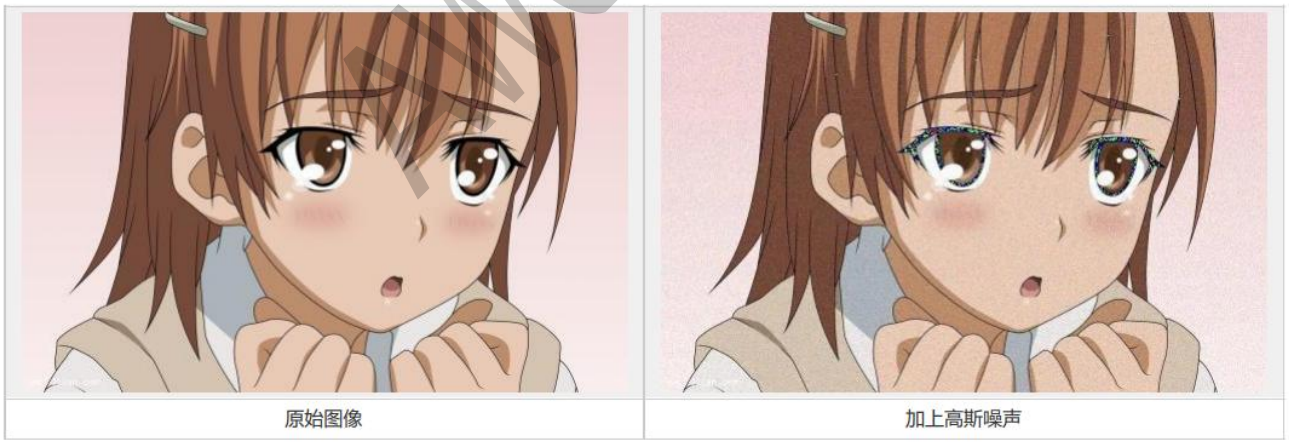


图 1.1 高斯噪声的概率密度函数

下面是图像加上高斯噪声的效果



原始图像　　加上高斯噪声

椒盐噪声和高斯噪声的对比如下图所示：

噪声种类	出现位置	灰度值
椒盐噪声	随机	基本是固定的（0 或 255）
高斯噪声	固定的（基本在每一点上）	噪声的幅值是随机的，噪点灰度

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

		值的概率密度函数服从高斯分布

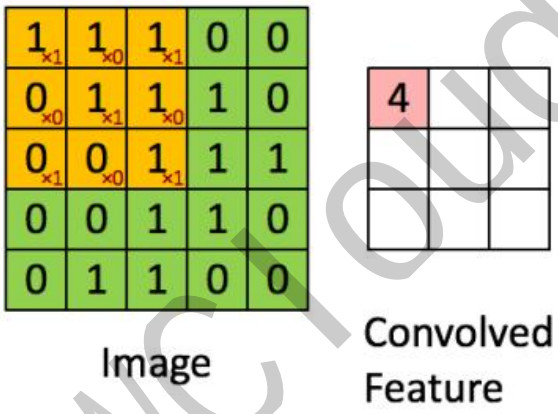
6.3.2 卷积和滤波

6.3.2.1 卷积和卷积核

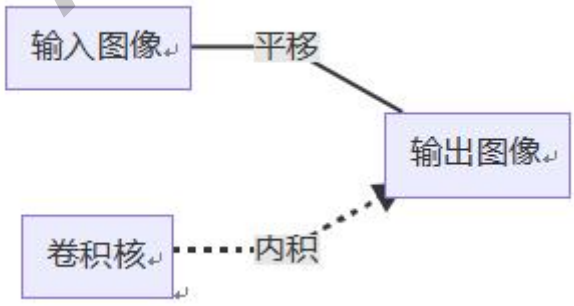
首先看下数学上卷积公式，可以理解为系统某一时刻的输出是由多个输入共同作用（叠加）的结果。

$$\int_{-\infty}^{\infty} f(\tau)g(x-\tau)d\tau$$

卷积应用在在图像处理时，f(x) 可以理解为原始像素点(source pixel)，所有的原始像素点叠加起来，就是原始图片了。g(x)可以称为作用点，所有作用 点合起来我们称为卷积核，卷积核上所有作用点依次作用于原始像素点后（即乘起来），线性叠加的输出结果，即是最终卷积的输出，也就是输出图片。



在图像处理中，卷积的过程如图所示，图像的像素矩阵和卷积核逐个相乘，得到输出像素值，算完之后，输入的像素框框再往右移一步继续计算，横向 计算完后，再往下移一步继续计算，过程可以简化为下图



在上面的动图中，输入图像是

AWCloud

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

卷积核是

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

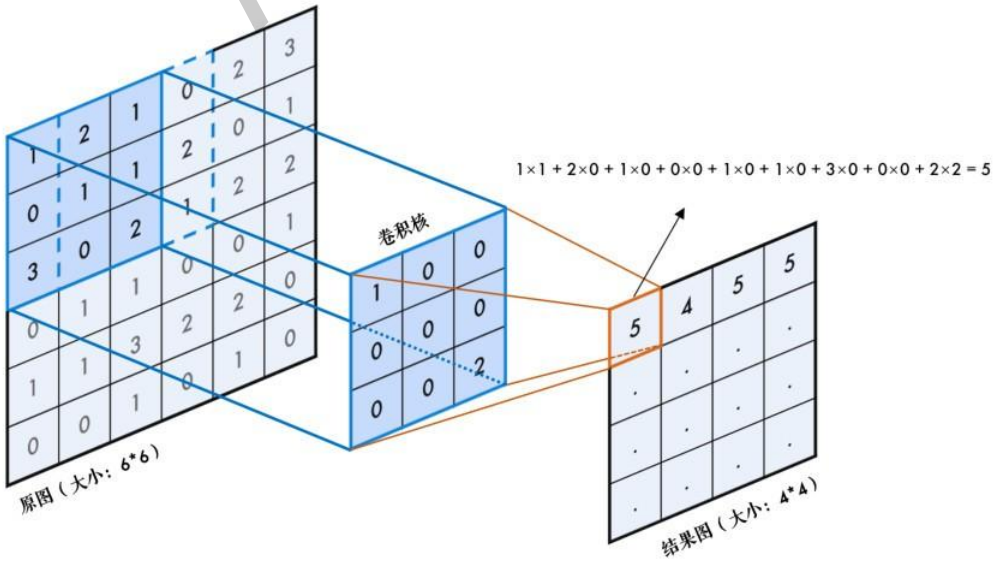
输出图像是

$$\begin{bmatrix} 4 & 3 & 4 \\ 2 & 4 & 3 \\ 2 & 3 & 4 \end{bmatrix}$$

输出图像中第一个元素 4 是怎么算的呢？由输入图像中 3×3 的区域与 3×3 的卷积核做内积得到：

$$4 = 1 \times 1 + 0 \times 1 + 1 \times 1 + 0 \times 0 + 1 \times 1 + 0 \times 1 + 1 \times 0 + 0 \times 0 + 1 \times 1$$

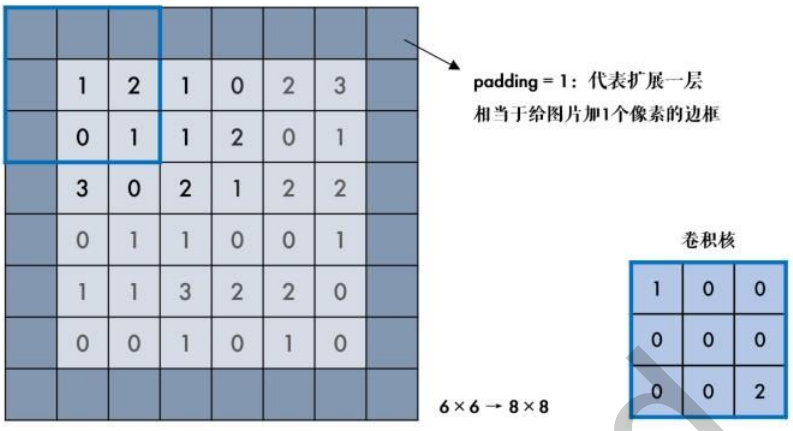
下图更直观地展示了卷积的计算过程。



AWCloud

在上图中我们用 3×3 的卷积核对一副 6×6 的图像进行卷积，得到的是 4×4 的图，输出图片比输入图片小，而在图像处理中，很多时候我们要求在对原始图 像做处理后不改变图像的大小，这个时候怎么办呢，我们可以把原图扩充一圈，再卷积，这个操作叫填充 padding。

在下图中，给 6x6 的图像加上边框后，就变成了 8x8 的图像，经过卷积后，输出图像是 6x6，和输入图像大小一致。



那我们加上的边框像素值用什么来填充呢，opencv 提供了几种类型的填充方式，下面我们详细介绍。

opencv 使用 cv2.copyMakeBorder(src,top, bottom, left, right ,borderType,value) 用来给图片添加边框，它有下面几个参数：

- src：原始图像
- top , bottom , left , right ：上下左右要扩展的像素数
- borderType ：边框类型，有 5 种类型，如下表所示

borderType	填充方式
BORDER_REFLICATE	直接复制最边缘的像素填充，例如：aaaa|abcdefg|ggggg
BORDER_REFLECT	镜像法，即以最边缘的像素为对称轴,例如：fedcba|abcdefg|gfedec
BORDER_REFLECT_101	和上面类似 BORDER_REFLECT，但在反射时会把边界空开，例如：fedcb|abcdefg|fedec
BORDER_WRAP	以图像的左边界与右边界相连，上下边界相连。例如：cdefgh|abcdefgh|abcdefg
BORDER_CONSTANT	固定的像素值填充

其中镜像法和固定值方式最常用，我们将做详细介绍。

固定值填充：

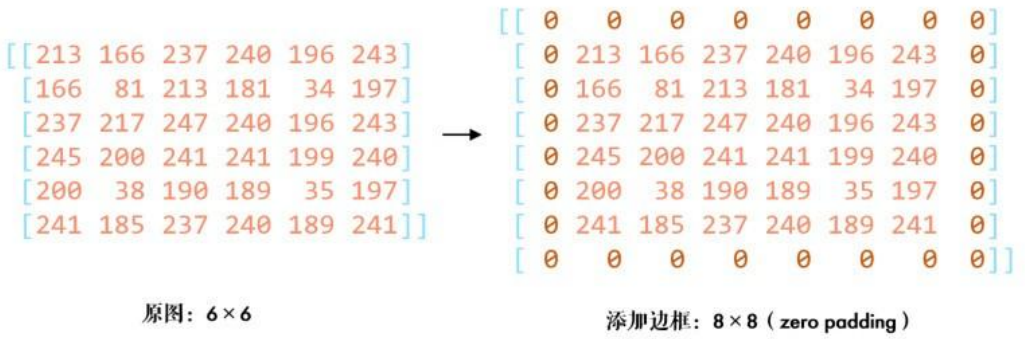
很好理解， cv2.BORDER_CONSTANT 这种方式就是边框都填充成一个固定的值，比如下面的程序都填充 0：

```
1.  img = cv2.imread('6_by_6.bmp', 0)
```

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

```
2.  # 固定值边框，统一都填充0 也称为zero padding
3.  cons = cv2.copyMakeBorder(img, 1, 1, 1, 1, cv2.BORDER_CONSTANT, value=0)
```

如下图，边界填充上固定值 0



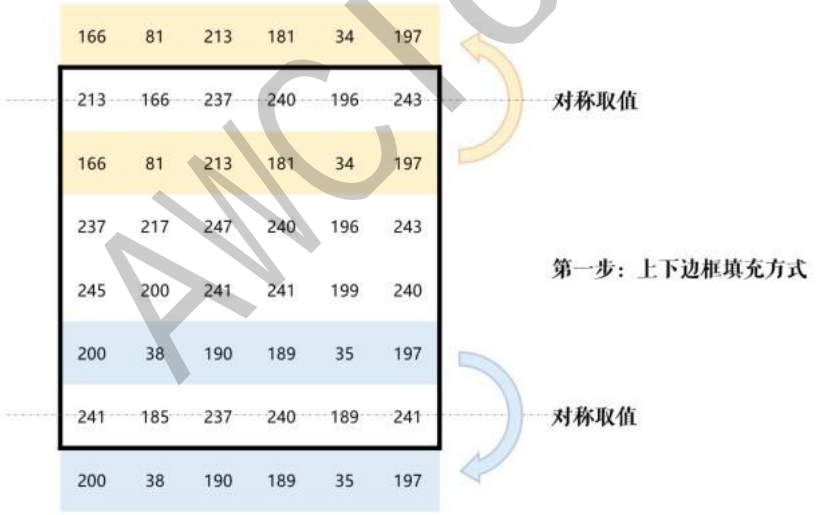
原图：6×6　　添加边框：8×8（zero padding）

镜像填充：

opencv 的默认填充方式是镜像填充，使用方法如下所示：

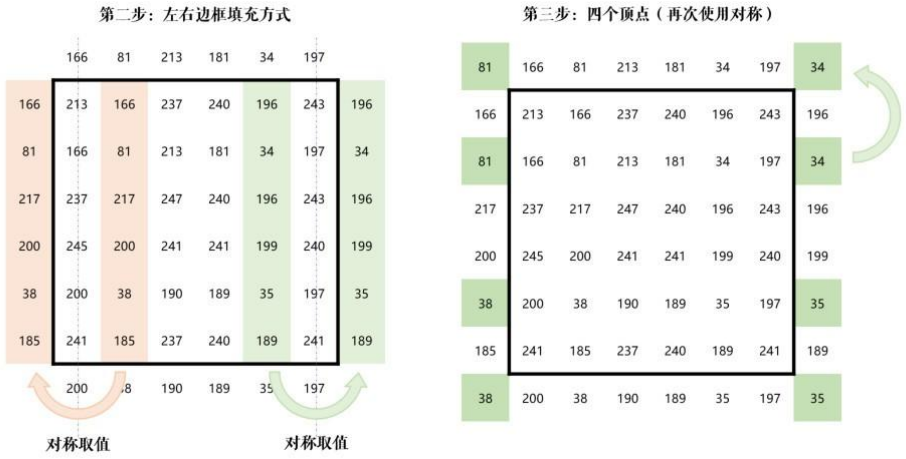
```
1.  img = cv2.imread('6_by_6.bmp', 0)
2.  default = cv2.copyMakeBorder(img, 1, 1, 1, 1, cv2.BORDER_DEFAULT)
```

首先进行上下填充，填充成与原图像边界对称的值，如下图：



同理再进行左右两边的填充，最后把四个顶点补充上就好了：

AWCloud



6.3.3 均值滤波

均值滤波对目标像素及周边像素取平均值后再填回目标像素，均值滤波的卷积核内的值大小相等，均值滤波的卷积核为：

$$kernel = \frac{1}{width \times height}\begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & 1 & \cdots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \cdots & 1 \end{bmatrix}$$

以一个 3×3 的均值滤波卷积核为例：

$$kernel = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

目标像素取周边 9 个（包括他自身）像素的像素值加权平均，由于卷积核内的值都为 1，所以每个像素的权重相等，相当于 9 个像素值相加再除以 9，得 到 9 个像素的平均值再替换目标像素的值。

opencv 的均值滤波函数为 cv2.blur(src, ksize[, dst[, anchor[, borderType]]]) ，用法如下

```
1. img = cv2.imread('lena.jpg')
2. blur = cv2.blur(img, (3, 3)) # 均值模糊
```

- 参数 src ： 图片
- 参数 ksize ： 卷积核的宽和高（必须是奇数），用元组表示。
- 参数 borderType ： 边界像素填充方式，参考上面的边界填充知识点，默认是镜像填充

6.3.4 方框滤波

均值滤波对目标像素及周边像素取求和后再填回目标像素，方框滤波的卷积核内的值大小相等，方框滤波的卷积核为：

AWCloud

$$kernel = \alpha \begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & 1 & \cdots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \cdots & 1 \end{bmatrix} \quad \text{不同情况下 } \alpha = \begin{cases} \frac{1}{width \times height} & normalize == True \\ 1 & normalize == False \end{cases}$$

- 当 normalize = true 时，方框滤波就变成了均值滤波。也就是说，均值滤波是方框滤波归一化（normalized）后的特殊情况。
- 当 normalize = false 时，为非归一化的方框滤波，此时 ，用于计算每个像素邻域内的积分特性。

以一个 3×3、 normalize = false 的方框滤波卷积核为例：

$$kernel = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

目标像素的值等于周边 9 个像素（包括自身）的像素和。

opencv 的均值滤波函数为 cv2.boxFilter(src, ddepth, ksize [, dst [, anchor [, normalize [, borderType]]]])，用法如下

```
1.  img = cv2.imread('lena.jpg')
2.  blur = cv2.boxFilter(img, -1, (3, 3), normalize=True)
```

- 参数 src ： 图片
- 参数 ddepth ： 输出图像的深度，-1 表示使用输入图像的深度
- 参数 ksize ： 卷积核的宽和高（必须是奇数），用元组表示。
- 参数 normalize ： 归一化标志，false 时 ， true 时为均值滤波。
- 参数 borderType ： 边界像素填充方式，参考上面的边界填充知识点，默认是镜像填充

6.3.5 中值滤波

中值滤波法是非线性滤波，将图像的每个像素用邻域 (以当前像素为中心的正方形区域)像素的中值代替。可以有效去除孤立的噪点，适用于去除椒盐噪 声和斑点噪声。由于是非线性滤波，所以计算起来比其他滤波器要慢。

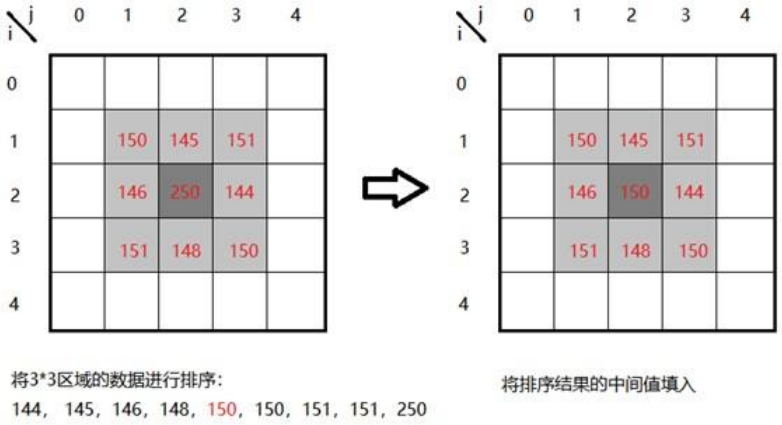
以一个 3 x 3 的像素窗为例，计算以点 [i,j] 为中心的函数窗像素中值步骤如下：

1. 按强度值大小排列像素点（排序结果为 144, 145, 146, 148, 150, 150, 151, 151, 250）

2. 选择排序像素集的中间值作为点 [i,j] 的新值（ [i,j] 原来的像素值 250 用排序结果的中值 150 代替）

AWCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

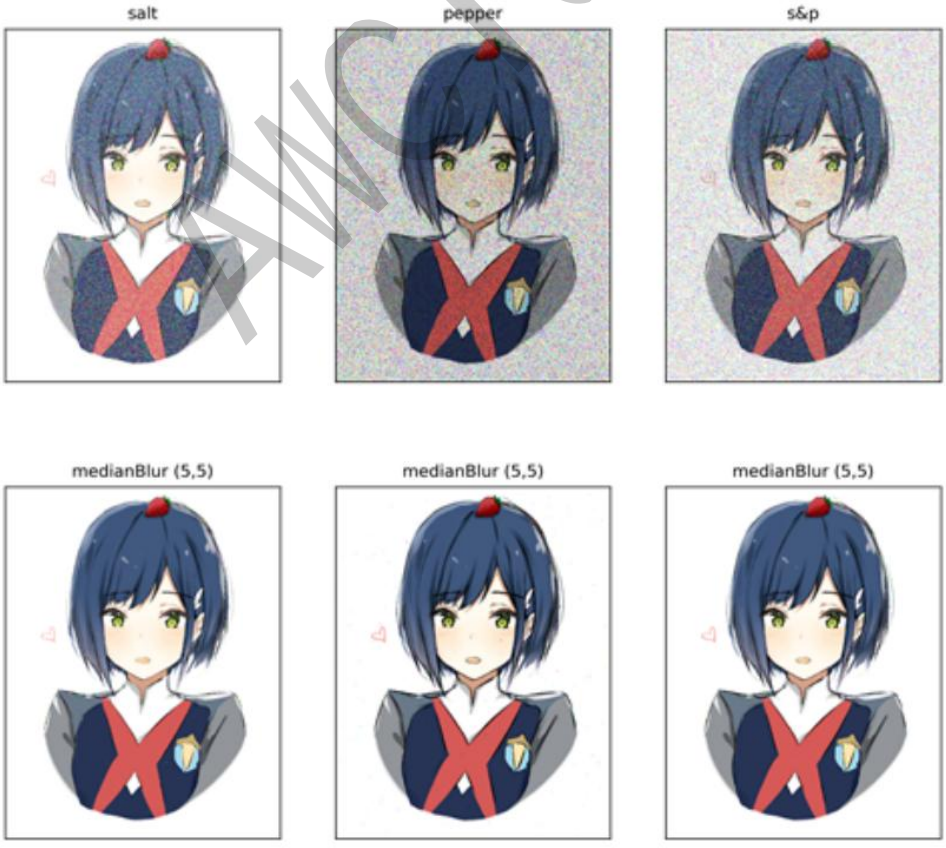


opencv 的中值滤波函数为 cv2.medianBlur(src, ksize[, dst])，用法如下

```
1. img = cv2.imread('salt_noise.bmp', 0)
2. median = cv2.medianBlur(img, 5) # 中值滤波
```

- 参数 src：图片
- 参数 ddepth：输出图像的深度，-1 表示使用输入图像的深度
- 参数 ksize：像素窗的宽和高（必须是奇数），用元组表示。

中值滤波对椒盐噪声的效果如下图所示。



AWCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

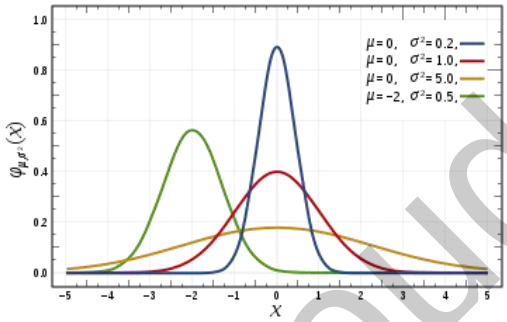
6.3.6 高斯滤波

卷积核的每个值相当于该值相对应的像素值的权重，中值滤波和均值滤波卷积核内的每个值相等，也就是说图像区域中每个像素的权重也就一样，高斯 滤波的卷积核权重是高斯分布的，越接近中心点像素的权重越高。

首先先看一维的高斯函数：

$$G(x) = \frac{1}{\sqrt{2\pi}\sigma} exp(-\frac{(x-\mu)^2}{2\sigma^2})$$

不同的 μ 和 σ 时的高斯分布图如下所示：



其中 μ 代表高斯函数对称中心的位置，也就是最高点的横坐标， σ 代表高斯分布的离散程度， σ 越大，数据分布越分散， σ 越小，数据分布越集中。从几 何上看， σ 越大，曲线越扁平，反之， σ 越小，曲线越瘦高。

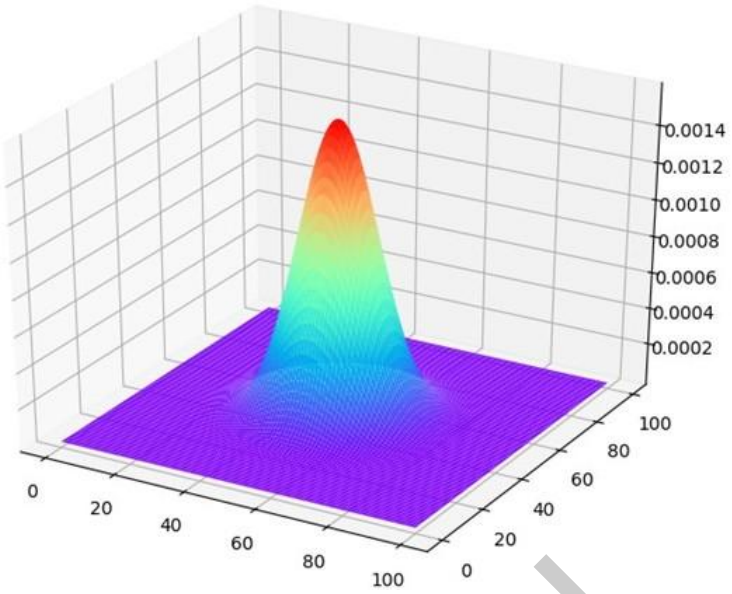
由于图像是二维，我们需要使用二维的高斯函数，二维的高斯函数其实就是互相独立的两个一维高斯函数相乘。

$$G(x,y) = \frac{1}{2\pi\sigma_x\sigma_y} exp(-\frac{(x-\mu_x)^2+(y-\mu_y)^2}{2\sigma_x\sigma_y}) = G(x)G(y)$$

二维的高斯函数分布图（ σ = 0）如下所示：

AWCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司



如何计算高斯核呢，以 3 × 3， σ = 0.8 的高斯核为例，以卷积核的中心位置为坐标原点进行取样。卷积核在各个位置的坐标，如下所示（x 轴水平向右， y 轴竖直向上）。

(-1,1)	(0,1)	(1,1)
(-1,0)	(0,0)	(1,0)
(-1,-1)	(0,-1)	(1,-1)

将各个位置的坐标 x 和 y 值代入到高斯分布函数$G(x,y)$中，得到相应的值，9 个位置的值都计算出来后，得到下面的卷积核

$$\begin{bmatrix} 0.057118 & 0.12476 & 0.057118 \\ 0.12476 & 0.2725 & 0.12476 \\ 0.057118 & 0.12476 & 0.057118 \end{bmatrix}$$

上面的卷积核要做归一化后取整，归一化取整后的卷积核如下所示，这就是一个 3 × 3， σ = 0.8 的高斯滤波卷积核：

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

opencv 中对应函数为 cv2.GaussianBlur(src, ksize, sigmaX[, dst[, sigmaY[, borderType]]]) ：

```
1.  img = cv2.imread('gaussian_noise.bmp')
```

AWCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

```
# 均值滤波 vs 高斯滤波
blur = cv2.blur(img, (5, 5)) # 均值滤波
gaussian = cv2.GaussianBlur(img, (5, 5), 1) # 高斯滤波，sigmaX = sigmaY = 1
```

- 参数 src ： 图片
- 参数 ksize ： 高斯核的宽和高（必须是奇数），用元组表示。
- 参数 sigmaX ： 水平方向的 σ，值越大说明高斯函数越平缓，则滤波出来的图片越模糊， sigmaX 为 0 时，它的值将由 ksize.width 和 ksize.heigh 自动确定
- 参数 sigmaY ： 竖直方向的 σ，如果 sigmaY 为零，则等于 sigmaX
- 参数 borderType ： 边界像素填充方式，参考上面的边界填充知识点，默认是镜像填充

6.3.7 滤波器比较

不同方式滤波器的特点如下表所示。

滤波器种类	基本原理	特点
均值滤波	使用模板内所有像素的平均值代替模板中心像素灰度值	不能很好地保护图像细节，在图像去噪的同时也破坏了图像的细节，从而使图 像变得模糊，不能很好地去除椒盐噪声
中值滤波	计算模板内所有像素中的中值，并用所计算出来的中值 体改模板中心像素的灰度值	对噪声不是那么敏感，去除斑点和椒盐噪声优先使用中值滤波，但是容易导致 图像的不连续性
高斯滤波	对图像邻域内像素进行平滑时，邻域内不同位置的像素 被赋予不同的权值	对图像进行平滑的同时，同时能够更多的保留图像的总体灰度分布特征

7 机械臂控制

在缺陷检测过程中，识别到缺陷，需要通知机械臂进行抓取动作，将识别到的缺陷铝片捡取出来

7.1 机械臂单个舵机控制

控制单个总线舵机对应的 API 为：

Arm_serial_servo_write(id, angle, time)

函数功能：控制总线舵机要运行到的角度。

参数解释：

id：要控制的舵机的 ID 号，范围是 1~6，每个 ID 号表示一个舵机，从最底端的舵机的 ID 为 1，往上依次增加，最上面的舵机 ID 为 6。

angle：控制舵机要运行到的角度，除了 5 号舵机（ID=5），其他舵机的控制范围都是 0~180，5 号舵机的控制范围是 0~270。

AWCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

time：控制舵机运行的时间，在有效范围内，舵机转动相同的角度，输入运行的时间越小，舵机运动越快。输入 0 则舵机以最快速度运行。

返回值：无。

代码如下：

```
import time
from Arm_Lib import Arm_Device

#    创建机械臂对象
Arm = Arm_Device()
time.sleep(.1)

#    单独控制一个舵机运动到某个角度
id = 6
Arm.Arm_serial_servo_write(id,    90, 500)
time.sleep(1)
```

7.2 机械臂多个舵机同时控制

一次控制 6 个总线舵机对应的 API 为：

Arm_serial_servo_write6(S1, S2, S3, S4, S5, S6, time)

函数功能：同时控制机械臂的六个舵机要运动到的角度。

参数解释：

S1：1 号舵机的角度值 0~180。

S2：2 号舵机的角度值 0~180。

S3：3 号舵机的角度值 0~180。

S4：4 号舵机的角度值 0~180。

S5：5 号舵机的角度值 0~270。

S6：6 号舵机的角度值 0~180。

time：控制舵机运行的时间，在有效范围内，舵机转动相同的角度，输入运行的时间越小，舵机运动越快。输入 0 则舵机以最快速度运行。

返回值：无。

代码如下：

```
import   time
from   Arm_Lib import Arm_Device

#    创建机械臂对象
Arm   = Arm_Device()
time.sleep(.1)

#    同时控制六个舵机运动，逐渐变换角度。
def   ctrl_all_servo(angle, s_time = 500):
```

AWCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

```
    Arm.Arm_serial_servo_write6(angle,   180-angle, angle, angle, angle, angle, s_time)
    time.sleep(s_time/1000)


def   main():
    dir_state = 1
    angle = 90

    # 让舵机复位归中
    Arm.Arm_serial_servo_write6(90, 90, 90,   90, 90, 90, 500)
    time.sleep(1)
    while True:
        if dir_state == 1:
            angle += 1
            if angle >= 180:
                dir_state = 0
        else:
            angle -= 1
            if angle <= 0:
                dir_state = 1
        ctrl_all_servo(angle, 10)
        time.sleep(10/1000)

try   :
    main()
except   KeyboardInterrupt:
    print(" Program closed! ")
    pass

del   Arm  # 释放掉 Arm 对象
```

7.3 机械臂单个舵机位置读取

在机械臂操作时，有时需要知道每个舵机当前对应的位置信息，因此提供了舵机当前位置读取接口

读取单个总线舵机的角度对应的 API 为：

Arm_serial_servo_read(id)

函数功能：读取总线舵机当前的角度值。

参数解释：

id：要读取的舵机的 ID 号，范围是 1~6，每个 ID 号表示一个舵机，从最底端的舵机的 ID 为 1，往上依次增加，最上面的舵机 ID 为 6。

返回值：对应 ID 舵机当前的角度，ID=5 时，角度范围为 0~270，其他都为 0~180。

代码如下：

```
import   time
from   Arm_Lib import Arm_Device
```

AWCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

```
3.
4.  #    创建机械臂对象
5.  Arm    = Arm_Device()
6.  time.sleep(.1)
7.
8.  #    单独控制一个舵机运动后，再读取它的角度
9.  id    = 6
10. angle    = 150
11. Arm.Arm_serial_servo_write(id,    angle, 500)
12. time.sleep(1)
13.
14. aa    = Arm.Arm_serial_servo_read(id)
15. print(aa)
16. time.sleep(.5)
17.
18. del    Arm   # 释放掉 Arm 对象
```

7.4 机械臂吸盘控制

针对铝片这类物品的抓取，无法使用机械臂抓手进行抓取，需要选用吸盘来进行吸附抓取，这里对吸盘的控制使用做个介绍

吸盘正确连接到控制平台后，识别到缺陷铝片，控制平台发送对应的串口信号控制吸盘进行吸取和释放动作：

```
1.  import serial
2.
3.  # 吸盘通过 USB 接口连接到控制平台
4.  # 指定吸盘在控制平台对应设备名称以及波特率
5.  com = serial.Serial('/dev/ttyUSB0', 115200)
6.
7.  # 2500 表示最大吸取，1500 表示释放
8.  string = '#000P2500T0000!'
9.  com.write(string.encode('UTF-8'))
```

8 Sqlite 使用

SQLite 是一个软件库，实现了自给自足的、无服务器的、零配置的、事务性的 SQL 数据库引擎。SQLite 是在世界上最广泛部署的 SQL 数据库引擎。SQLite 源代码不受版权限制。

在缺陷检测时，需要对相关的缺陷信息进行记录保存，因此这里选用了 sqlite

Sqlite 相关操作如下：

```
1.  import sqlite3
2.
3.  # 创建数据库
4.  conn = sqlite3.connect('ggg.db')
5.  cursor = conn.cursor()
6.
7.  # 创建表，包含铝板 uuid、铝板缺陷名称、铝板图片保存路径、铝板缺陷检测时间、铝板缺陷识
    别准确率以及改条数据创建时间，其中 uuid、路径和缺陷名为唯一约束，id 为自增主键
```

AWCloud

```
cursor.execute("create table if not exists defect_list (id integer primary key, uuid char, path char(64), name char, prediction_time char, score char, [CreatedTime] TimeStamp NOT NULL DEFAULT (datetime('now','localtime')), UNIQUE(uuid, path, name))")

# insert 操作
uid = "awsdqwe"
cursor.execute("insert or ignore into defect_list(uuid, path, name, prediction_time, score) values(?, ?, ?, ?, ?)", (uid, None, None, None, None))

# select 操作
cursor.execute("select count(distinct uuid) from defect_list")
values1 = cursor.fetchall()
count = values1[0][0]
print("count is {}".format(count))

# update 操作
name = "ccccccc"
id1 = 1
cursor.execute("update defect_list set name='{}' where id='{}'".format(name, id1))

# delete 操作
id2 = 1
cursor.execute("delete from defect_list where id='{}'".format(id2))
conn.commit()
```

9 对外接口封装

由于控制平台需要显示缺陷检测中的相关信息，因此需要提供对应的接口供前端页面访问，这里使用 Flask 来提供接口

Flask 是目前十分流行的 web 框架，采用 Python 编程语言来实现相关功能。它被称为微框架 (microframework)，“微”并不是意味着把整个 Web 应用放入到一个 Python 文件，微框架中的“微”是指 Flask 旨在保持代码简洁且易于扩展，Flask 框架的主要特征是核心构成比较简单，但具有很强的扩展性和兼容性，程序员可以使用 Python 语言快速实现一个网站或 Web 服务。

下面用个简单的例子来介绍 flask 的使用，代码如下：

```
from flask import Flask  # 导入 flask 库

app = Flask(__name__)  #  flask 类对象初始化

@app.route('/hello', methods=['GET'])  # 路由
def hello():
    return "Hello World!\n"     # 返回 Hello World

if __name__ == "__main__":
    app.run(host="0.0.0.0", port="80") # 启动服务器（指定端口 80）
```

执行下面的命令启动服务

AWCloud

版权所有 © 重庆海云捷迅科技有限公司

```
python server.py
```

```
root@awcloud-NUC8i5BEH:/opt/MVS/Samples/64/GrabImage# python3 server.py
 * Serving Flask app "server" (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployment.
   Use a production WSGI server instead.
 * Debug mode: off
 * Running on http://0.0.0.0:80/ (Press CTRL+C to quit)
127.0.0.1 - - [05/Jul/2021 17:41:08] "GET /hello HTTP/1.1" 200 -
172.16.18.163 - - [05/Jul/2021 17:41:24] "GET /hello HTTP/1.1" 200 -
```

执行下面的命令可以访问对应的接口，获取返回信息

```
curl -X GET http://127.0.0.1:80/hello
```

```
root@awcloud-NUC8i5BEH:/home/awcloud#
root@awcloud-NUC8i5BEH:/home/awcloud# curl -X GET http://172.16.18.163:80/hello
Hello World!
root@awcloud-NUC8i5BEH:/home/awcloud#
```

10 基于 VUE 的应用开发

10.1 VUE 基础实验

1.Vue 中条件控制指令：v-for、v-if

v-for 指令是用来迭代遍历的，可以遍历数组、对象等列表变量。

```
<div id="head">
    <!--遍历array1，将值放入arr中-->
    <p v-for="arr in array1">{{arr}}</p>
</div>
<script>
    let vue1 = new Vue({
        el: "#head",
        data: {
            array1: ['a', 'b', 'c', 1, 2, 3]
        }
    });
</script>
```

如上所示，通过 v-for 遍历之后，会把 array1 的数组用 展示出来，其 html 代码就会变成：

a

b

c

AWCloud

1

2

3

一般在将后台返回的列表数据在前端用表格展示时使用。

v-if 是控制 html 元素如何渲染的判断。举例：

```
<VBorder class="main-center-item" :title="'检测结果'">
  <span slot="title" class="tip" v-if="testingImg" >({{ testingText }})</span>
  <div  class="detection-result">
    <img class="image" v-if="testingImg" :src="testingImg" />
    <span v-if="!testingImg" class="text">未检测到缺陷</span>
  </div>
</VBorder>
</div>
```

当 testingImg 变量返回 true 时，div 代码块会加载图片到页面。如果为 false，则会显示文本。

2.Vue 中绑定表单指令：v-model

V-model 负责监听用户的输入事件，从而更新数据，并对一些极端场景进行一些特殊处理 。当输入事件发生时，实时更新 vue 实例中的数据

```
<div class="footer-item">
  <span class="label">抓取位置：</span>
  <el-slider v-model="grabPosition" @change="setConfig" :max="180" :color="'#167aff'"></el-slider>
  <span class="slider-value">{{grabPosition}}</span>
</div>
```

如上所示 v-model=”grabPosition” @change="setConfig" :max="180" :color="'#167aff'"，可以根据配置文件中的数据进行变更，即配置文件数据发生变化时，页面的显示也会随着改变

Vue 的组件 components 和模板 template

Vue 的组件模板功能为前端良好的代码书写风格提供了可能，虽然不使用组件和模板也可以写前端代码，但是可能会存在大量的重复代码。因此可以通过抽象一个组件：

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

```
<template>
  <div class="border">
    <h5 class="title"><span class="icon"></span>{{ title }}<slot name="title"></slot></h5>
    <div class="centent">
      <slot></slot>
    </div>
  </div>
</template>
<script>
export default {
  name: "VBorder",
  props: {
    title: String
  }
};
```

抽象时需要使用模板 template 的关键字，对应的该模板中的页面元素、属性、方法事件、css 样式等都可以统一管理。在后续使用时，只需要 import 这个组件，就可以使用了：

```
import VHeader from "./components/Header.vue";
import VBorder from "./components/Border.vue";
import * as echarts from "echarts/lib/echarts";
import "echarts/lib/chart/line";
import "echarts/lib/chart/bar";
import "echarts/lib/chart/pie";
```

```
<VBorder class="main-left-top" :title="'检测区域'">
  <div class="detection-result">
    <img :src="config.suckerIp + '/img'" class="image detection" />
  </div>
</VBorder>
```

如上图所示，只需要简单的几行代码就可以应用该组件了，如果需要修改，只需要在组件 vue 文件中统一修改就可以了，这样使得代码维护也变得非常简单。

10.2 VUE 接口调用

1.使用 axios 访问 API

axios 是一个基于 Promise 用于浏览器和 nodejs 的 HTTP 客户端，它本身具有以下特征：

- 从浏览器中创建 XMLHttpRequest
- 支持 Promise API
- 拦截请求和响应
- 转换请求和响应数据
- 取消请求

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

- 自动转换 JSON 数据
- 客户端支持防止 CSRF/XSRF

本项目 src/js/axios.js 例子，先引入 axios 模块：

```
import axios from "axios";
axios.interceptors.request.use(config => {
  return config;
});
axios.interceptors.response.use(res => {
  if (res.status === 200) {
    return res.data;
  }
});
export default axios;
```

在 src/js/api.js 中引入上述文件，根据接口文档实现调用函数，具体实现请参考接口文档：

```
import Axios from "./axios";

export const GetConfig = () => {
  const url = "/config.json";
  return Axios.get(url, {});
};
```

2.REST API 接口规范简介

REST（Representational State Transfer）表述性状态转换，REST 指的是一组架构约束条件和原则。如果一个架构符合 REST 的约束条件和原则，我们就称它为 RESTful 架构。REST 本身并没有创造新的技术、组件或服务，而隐藏在 RESTful 背后的理念就是使用 Web 的现有特征和能力，更好地使用现有 Web 标准中的一些准则和约束。

REST 规范中利用标准的 HTTP 方法对应资源的（增查改删）CRUD：

- GET：查询

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司

- POST：创建单个资源。POST 一般向“资源集合”型 uri 发起。
- PUT：更新单个资源（全量），客户端提供完整的更新后的资源。与之对应的是 PATCH，PATCH 负责部分更新，客户端提供要更新的那些字段。PUT/PATCH 一般向“单个资源”型 uri 发起。
- DELETE：删除

安全性和幂等性

- 安全性：不会改变资源状态，可以理解为只读的；
- 幂等性：执行 1 次和执行 N 次，对资源状态改变的效果是等价的。

常用的 http 状态码及使用场景

状态码	使用场景
400 bad request	常用在参数校验
401 unauthorized	未经验证的用户，常见于未登录。如果经过验证后依然没权限，应该 403（即 authentication 和 authorization 的区别）。
403 forbidden	无权限
404 not found	资源不存在
500 internal server error	非业务类异常
503 service unavaliable	由容器抛出，自己的代码不要抛这个异常

AWCloud

海云捷迅专有和保密信息
版权所有 © 重庆海云捷迅科技有限公司